İSTANBUL OKAN ÜNİVERSİTESİ · MESLEK YÜKSEKOKULU · 2026–2027 GÜZ

MBBO205 Bulut Bilişim ve Yapay Zekâ

Hafta 3 Ders Notu: Bash betikleme ve komut satırında veri

Veri yapıları, algoritma karmaşıklığı, boru hatları, grep, sed, awk, jq ve cron

Dr. Öğr. Üyesi Hasan Oğuz

Bu haftanın hedefleri. Hafta sonunda öğrenci (i) temel veri yapılarını ve erişim maliyetlerini tanır; (ii) O-notasyonuyla algoritma büyüme hızlarını karşılaştırır; (iii) standart akışları, yönlendirmeyi ve boru hattını kullanır; (iv) grep, sed, awk ve jq ile gerçek bir web sunucusu günlüğünü ve CSV/JSON verisini çözümler; (v) güvenli bir Bash betiği yazar ve cron ile zamanlar.

İlgili kazanımlar: DÖK2, DÖK3 **Süre:** 1 saat teori + 1 saat uygulama

İçindekiler

1 Veri yapıları **2**

2 Algoritma karmaşıklığı **2**

2.1 O-notasyonu . . . 2

2.2 Beş karmaşıklık sınıfı . . . 2

2.3 İkili arama . . . 3

2.4 Aynı soru, iki çözüm . . . 3

3 Unix felsefesi ve akışlar **3**

4 Bash betikleri **4**

4.1 Anatomi . . . 4

4.2 Metin araçları . . . 4

4.3 cron . . . 5

5 Uygulama: access.log çözümlemesi **5**

5.1 Alanları tanıyın . . . 5

5.2 Soru 1: Kim en çok istek atıyor? . . . 5

5.3 Soru 2: Durum kodları . . . 6

5.4 Soru 3: Model API'sinin yanıt süresi ve yoğun saatler . . . 6

5.5 CSV temizliği . . . 6

5.6 JSON ve jq . . . 6

5.7 cron ile zamanlama . . . 7

6 Laboratuvar görevi **7**

6.1 Ana görev: `log_raporu.sh` 7

6.2 Bozuk sistem görevi: Windows'tan gelen betik 7

6.3 Teslim listesi 8

7 Kendinizi sınayın **8**

1 Veri yapıları

Veriyi nasıl tuttuğunuz, ona ne kadar hızlı erişebileceğinizi belirler.

Yapı	Özellik	YZ ve veri işlemede örnek
Dizi (array)	Bitişik bellek; indeksle $O(1)$ erişim	NumPy dizileri, tensörler
Bağlı liste	Ekleme ucuz, arama $O(n)$	Kuyruk ve yığınların temeli
Hash tablosu	Anahtar $\rightarrow$ değer; ortalama $O(1)$	Python `dict`, awk dizileri, sözcük sayımı
Ağaç	Hiyerarşi; dengeli ağaçta arama $O(\log n)$	Dizin yapısı, veritabanı indeksi, karar ağacı
Graf	Düğümler ve kenarlar	Sosyal ağ, yol bulma, sinir ağı hesap grafı
Yığın / kuyruk	LIFO / FIFO	Geri alma işlemi, iş kuyruğu

2 Algoritma karmaşıklığı

2.1 O-notasyonu

Bir algoritmanın çalışma süresi $T(n)$, girdi boyutu n'nin fonksiyonudur. Bizi sabitler değil, n büyüdüğünde sürenin nasıl büyüdüğü ilgilendirir:

$$T(n) = O\big(f(n)\big) \iff \exists\, c > 0,\ n_0 :\ T(n) \le c\, f(n) \quad \forall n \ge n_0. \tag{1}$$

Örneğin $T(n) = 3n^2 + 5n + 7$ için yeterince büyük n'de n^2 terimi baskındır: $T(n) = O(n^2)$.

2.2 Beş karmaşıklık sınıfı

Sınıf	Örnek	$n = 10^6$ için adım
$O(1)$	Hash tablosunda anahtar arama	1
$O(\log n)$	Sıralı dizide ikili arama	≈ 20
$O(n)$	Dosyayı bir kez okumak (`awk`, `wc`)	10^6
$O(n \log n)$	Sıralama (`sort`)	$\approx 2 \times 10^7$
$O(n^2)$	Her satırı her satırla karşılaştırmak	10^{12}

Çözümlü örnek. Saniyede 10^9 basit işlem yapan bir işlemci varsayalım. $n = 10^6$ satır için $O(n \log n)$ bir algoritma yaklaşık $2 \times 10^7 / 10^9 = 0{,}02$ s sürer. $O(n^2)$ bir algoritma ise $10^{12}/10^9 = 1000$ s ≈ 17 dakika sürer. Veri 10 kat büyürse ilki yaklaşık 12 kat, ikincisi 100 kat yavaşlar. Bu fark daha hızlı donanımla kapatılamaz.

2.3 İkili arama

Sıralı bir listede ortadaki elemana bakılır; aranan değer küçükse sol yarıda, büyükse sağ yarıda aramaya devam edilir. Her adımda aday sayısı yarıya iner:

$$\frac{n}{2^k} = 1 \;\Rightarrow\; k = \log_2 n, \qquad \log_2 10^6 \approx 19{,}9. \tag{2}$$

Bir milyon kayıtta en fazla 20 karşılaştırma yeter. Ön koşul verinin sıralı olmasıdır; sıralamanın bedeli $O(n \log n)$'dir. Veritabanı indeksleri (hafta 7) bu fikrin ağaç biçimidir.

2.4 Aynı soru, iki çözüm

"Her IP adresi kaç istek attı?" sorusu iki biçimde çözülebilir:

- `sort | uniq -c`: önce sırala ($O(n \log n)$), sonra ardışık aynı satırları say. `uniq` yalnızca yan yana duran satırları görür; sıralanmamış girdide yanlış sayar.
- `awk '{c[$1]++}'`: her satırda hash tablosunda bir sayacı artır ($O(n)$); sıralama yalnızca az sayıdaki sonuç için yapılır.

20.000 satırda fark hissedilmez; milyarlarca satırda belirleyici olur.

3 Unix felsefesi ve akışlar

Unix araçları üç ilkeye dayanır: her araç tek bir işi iyi yapar; araçlar birbirine düz metin satırlarıyla veri aktarır; boru hattı (`|`) küçük araçları bir veri işleme hattına dönüştürür. Her sürecin üç standart akışı vardır:

No	Akış	Yönlendirme	Örnek
0	stdin	`<`	`wc -l < access.log`
1	stdout	`>`, `>>`	`ls > liste.txt`
2	stderr	`2>`	`find / -name x 2> /dev/null`
1+2	ikisi	`> dosya 2>&1`	`betik.sh > log.txt 2>&1`
–	boru hattı	`|`	`grep 404 access.log | wc -l`
–	çatal	`tee`	`rapor.sh | tee rapor.txt`

`cron` işlerinde `2>&1` kritiktir: hata mesajları aksi halde kaybolur.

4 Bash betikleri

4.1 Anatomi

```
#!/usr/bin/env bash                # shebang: hangi yorumlayıcı?
set -euo pipefail                  # hata, tanımsız değişken, boru hatası hatasında dur
LOG ="${1:?kullanım: $0 access.log}"  # zorunlu argüman
echo "Toplam istek: $(wc -l < "$LOG")"
for kod in 200 404 500; do
  n =$(awk -v k ="$kod" '$9 == k' "$LOG" | wc -l)
  echo "HTTP $kod: $n"
done
```

Bu betik `access.log` üzerinde şu çıktıyı verir: `Toplam istek: 20000`, `HTTP 200: 18164`, `HTTP 404: 1030`, `HTTP 500: 198`. Argümansız çalıştırılırsa hata mesajı basar ve çıkış kodu 1 olur.

- `$1`, `$2` konumsal argümanlar; `$#` argüman sayısı; `$@` tümü.
- `$?` son komutun çıkış kodu: 0 başarı, 0 dışı hata. `cron` ve CI sistemleri buna bakar.
- `set -e` olmadan betik hatadan sonra sessizce devam eder ve yarım dosyalar üretir; veri hatlarında en tehlikeli hata biçimidir.
- Değişkenleri her zaman tırnak içinde kullanın: `"$LOG"`. Boşluk içeren dosya adları aksi halde bölünür.

4.2 Metin araçları

Komut	Görevi	Örnek
`grep`	Desene uyan satırları süz	`grep " 500 " access.log`
`cut`	Sütun kes	`cut -d, -f3 satislar.csv`
`sort`	Sırala (`-n` sayısal, `-r` ters)	`sort -rn`
`uniq -c`	Ardışık aynıları say	`sort | uniq -c`
`sed`	Bul ve değiştir	`sed 's/NA//g'`
`awk`	Alan tabanlı küçük programlar	`awk '{print $1}'`
`jq`	JSON sorgula	`jq '.[].model'`
`wc -l`	Satır say	`wc -l access.log`

`awk` kendi başına bir programlama dilidir [1]: alanlar `$1`, `$2` ..., son alan `$NF`, satır numarası `NR`, alan ayırıcı `-F`.

4.3 cron

`cron` beş zaman alanı ve bir komuttan oluşan satırlarla işleri zamanlar: dakika, saat, ayın günü, ay, haftanın günü.

İfade	Anlamı
`0 * * * *`	Her saat başı
`*/15 * * * *`	Her 15 dakikada bir
`30 2 * * 1`	Her pazartesi 02:30

`cron` kısa bir `PATH` ile çalışır; komut ve dosyalarda tam yol kullanılmalıdır (Hafta 1'deki PATH görevinin doğrudan uygulaması).

5 Uygulama: access.log çözümlemesi

Ders klasöründeki `access.log` dosyası 20.000 satırlık sentetik bir web sunucusu günlüğüdür. Aşağıdaki tüm çıktılar bu dosya üzerinde gerçekten çalıştırılarak alınmıştır.

5.1 Alanları tanıyın

```
$ head -1 access.log
10.0.7.121 - - [12/Oct/2026:00:03:06 +0300] "GET /index.html HTTP/1.1" 200 1596 "-"
↪  "Mozilla/5.0 (X11; Linux x86_64)" 0.085
$ head -1 access.log | tr " " "\n" | nl | sed -n '1p;4p;6,7p;9,10p'
     1  10.0.7.121
     4  [12/Oct/2026:00:03:06
     6  "GET
     7  /index.html
     9  200
    10  1596
```

Son alan (`$NF`) saniye cinsinden yanıt süresidir.

5.2 Soru 1: Kim en çok istek atıyor?

```
$ awk '{print $1}' access.log | sort | uniq -c | sort -rn | head -5
   3996 10.0.6.125
   1904 10.0.1.139
   1169 10.0.7.121
    883 10.0.3.250
    706 10.0.8.48
```

Tek bir IP günün yaklaşık %20'sini üretiyor: bir bot, yük testi ya da saldırı olabilir.

5.3 Soru 2: Durum kodları

```
$ awk '{c[$9]++} END {for (k in c) print c[k], k}' access.log | sort -rn
18164 200
1030 404
608 403
198 500
$ grep -c '"GET /admin' access.log
608
```

Tüm 403 yanıtları `/admin` yolundan geliyor: yetkisiz erişim denemeleri. 500 hataları yalnızca API uç noktalarında.

5.4 Soru 3: Model API'sinin yanıt süresi ve yoğun saatler

```
$ awk '$7 =="/api/predict" {s+ =$NF; n++} END {printf "%d istek, ort. %.3f s\n", n,
↪  s/n}' access.log
6785 istek, ort. 0.340 s
$ awk -F'[][]' '{split($2,a,":"); print a[2]}' access.log | sort | uniq -c | sort -rn |
↪  head -3
   2077 13
   1941 14
   1874 15
```

Yanıt süreleri sağa çarpıktır; ortalama yanıltıcı olabilir. 6. haftada yüzdelikleri (p95) göreceğiz.

5.5 CSV temizliği

```
$ wc -l satislar.csv
1226 satislar.csv
$ awk -F, 'NR>1 && $5 =="" ' satislar.csv | wc -l      # eksik adet
35
$ sort satislar.csv | uniq -d | wc -l                  # yinelenen satır
25
$ sed -E 's/, +/,/g; s/ +,/,/g' satislar.csv | awk -F, '!seen[$0]++' > temiz.csv
$ wc -l < temiz.csv
1201
```

`awk '!seen[$0]++'` yinelenenleri sıralamadan atar ve satır sırasını korur: yine bir hash tablosu, $O(n)$.

5.6 JSON ve jq

```
$ jq -r '.[] | select(.metrikler.dogruluk > 0.9) | "\(.deney) \(.model)
↪  \(.metrikler.dogruluk)"' deneyler.json
exp06 xgboost 0.923
exp07 xgboost 0.915
exp11 xgboost 0.902
$ jq 'map(.sure_sn) | add' deneyler.json
```

```
1311.9999999999998
$ jq -r 'sort_by(-.metrikler.f1)[0].deney' deneyler.json
exp05
```

Toplamın 1312 yerine $1311{,}9999\ldots$ çıkması bir kayan nokta yuvarlama etkisidir: $0{,}1$ gibi ondalık sayılar ikili tabanda tam gösterilemez. Para hesaplarında bu nedenle ondalık türler kullanılır (hafta 7, `NUMERIC`).

5.7 cron ile zamanlama

```
$ systemctl is-active cron
$ crontab -e
0 * * * * $HOME/mbbo205/hafta03/rapor.sh $HOME/mbbo205/hafta03/access.log >>
↪  $HOME/mbbo205/hafta03/cron.log 2>&1
$ crontab -l
$ tail -f ~/mbbo205/hafta03/cron.log
```

Test için `*/2 * * * *` kullanılıp iki dakika beklenebilir. WSL'de cron çalışmıyorsa: `sudo systemctl enable --now cron`.

6 Laboratuvar görevi

6.1 Ana görev: `log_raporu.sh`

Aşağıdaki iskeleti tamamlayın. Rapor şunları içermelidir: en çok istek atan 10 IP; durum kodu dağılımı ve 5xx oranı (%); `/api/predict` için ortalama ve en yüksek yanıt süresi; en yoğun 3 saat. Çıktı `rapor_TARIH.txt` dosyasına yazılır ve betik `cron`'a eklenir.

```
#!/usr/bin/env bash
set -euo pipefail
LOG ="${1:?kullanım: $0 access.log}"
OUT ="rapor_$(date +%F).txt"
{
  echo " == En çok istek atan 10 IP =="
  awk '{print $1}' "$LOG" | sort | uniq -c | sort -rn | head -10
  # ... diğer bölümleri siz ekleyin
} | tee "$OUT"
```

6.2 Bozuk sistem görevi: Windows'tan gelen betik

Bir arkadaşınız `temizle.sh` betiğini Windows'ta düzenleyip göndermiş:

```
$ chmod +x temizle.sh && ./temizle.sh
/usr/bin/env: 'bash\r': No such file or directory
$ file temizle.sh
temizle.sh: Bourne-Again shell script, ASCII text executable, with CRLF line terminators
$ cat -A temizle.sh | head -1
#!/usr/bin/env bash^M$
```

Görev: belirtiyi, hipotezleri ve testleri yazın; betiği düzeltin; `KOK_NEDEN_RAPORU.md` şablonunu doldurun; önleme için iki öneri verin.

İpucu: `^M` karakteri `\r` (satır başı) karakteridir. Windows satır sonu CRLF (`\r\n`), Linux satır sonu LF (`\n`) kullanır. Düzeltilen betik `satislar_temiz.csv` için 1191 satır raporlamalıdır.

6.3 Teslim listesi

- `log_raporu.sh` ve `rapor_TARIH.txt`
- `crontab -l` çıktısı
- `KOK_NEDEN_RAPORU.md` (CRLF)
- `temiz.csv` için satır sayısı

Son teslim: [tarih, saat] Teslim yeri: [LMS / e-posta]

7 Kendinizi sınayın

1. $T(n) = 5n \log_2 n + 200n$ fonksiyonunun O-notasyonundaki sınıfı nedir? Neden?
2. Sıralı bir telefon rehberinde 10 milyon kayıt var. İkili aramayla en fazla kaç karşılaştırmada bir adı bulursunuz?
3. `uniq -c access.log` komutu neden yanlış sonuç verir? Doğrusu nedir?
4. `komut > cikti.txt 2>&1` ile `komut 2>&1 > cikti.txt` arasındaki fark nedir?
5. `access.log` içinde her yol (`$7`) için 404 sayısını veren bir `awk` komutu yazın.
6. `set -u` bir betiği hangi hataya karşı korur? Örnek verin.
7. "Her iş günü 08:45'te" çalışacak bir cron ifadesi yazın.
8. `jq` ile `deneyler.json` içinden model türüne göre ortalama doğruluğu hesaplayın (ipucu: `group_by`).

Hafta 4 için hazırlık

Bir bulut sağlayıcıda öğrenci hesabı açın; kredi kartı gerekiyorsa bildirin (okul sunucusunda alternatif hazırlanacak). SSH anahtarınızın hazır olduğundan emin olun.

Kaynaklar

[1] Free Software Foundation, *The GNU Awk User's Guide*. `https://www.gnu.org/software/gawk/manual/gawk.html`

[2] GNU, *Bash Reference Manual*. `https://www.gnu.org/software/bash/manual/bash.html`

[3] jq 1.7 Manual. `https://jqlang.org/manual/`

[4] Linux man-pages, *crontab*(5). `https://man7.org/linux/man-pages/man5/crontab.5.html`

[5] T. H. Cormen, C. E. Leiserson, R. L. Rivest, C. Stein, *Introduction to Algorithms*, 4. baskı, MIT Press, 2022. `https://mitpress.mit.edu/9780262046305/introduction-to-algorithms/`

[6] MIT, *The Missing Semester*, "Data Wrangling." https://missing.csail.mit.edu/2020/data-wrangling/